

Automated Deep CNN-LSTM Architecture Design for Solar Irradiance Forecasting

Seyed Mohammad Jafar Jalali^{ID}, *Member, IEEE*, Sajad Ahmadian^{ID},
Abdollah Kavousi-Fard^{ID}, *Senior Member, IEEE*, Abbas Khosravi^{ID}, *Senior Member, IEEE*,
and Saeid Nahavandi^{ID}, *Fellow, IEEE*

Abstract—Accurate prediction of solar energy is an important issue for photovoltaic power plants to enable early participation in energy auction industries and cost-effective resource planning. This article introduces a new deep learning-based multistep ahead approach to improve the forecasting performance of global horizontal irradiance (GHI). A deep convolutional long short-term memory is used to extract optimal features for accurate prediction of the GHI. The performance of such deep neural networks directly depends on their architectures. To deal with this problem, a swarm evolutionary optimization method, called the sine-cosine algorithm, is applied and advanced to automatically optimize the network architecture. A three-phase modification model is proposed to increase the diversity of population and avoid premature convergence in the optimization mechanism. The performance of the proposed method is investigated using three datasets collected from three solar stations in the east of the United States. The experimental results demonstrate the superiority of the proposed method in comparison to other forecasting models.

Index Terms—Convolutional long short-term memory, deep neuroevolution, evolutionary computation, solar energy forecasting.

I. INTRODUCTION

SOLAR energy is one of the most widely spread types of renewable energy sources which has found its place in the competitive power market in recent years [1], [2]. The modern technology has made it possible for human to harness this free and sustainable source of energy in either electrical or thermal forms. Power grids have faced a very high growth of this class of energy in their structures as it has been recognized as the cleanest and most abundant renewable energy type available in the society [3], [4]. Some of the most significant advantages of solar energy in power grids can be named as reducing the emission and greenhouse gasses, mitigating the power losses

and costs, providing the possibility of direct charging of dc loads, such as laptops, electric vehicles, and phones without converters and enhanced power quality [5], [6]. Despite these promising characteristics, the high penetration of solar energy in the grid could cause severe instability issues. This is mainly due to the highly volatile and random nature of this type of energy [7]–[9]. Therefore, it is necessary to have a precise and accurate prediction of the solar energy to support the ever increasing role of this green energy in the future of power systems. In this regard, solar irradiance is considered as the most significant and critical parameter in determining the characteristics of the solar units, particularly for knowing that how much power might be finally harvested in a particular region [10], [11]. Therefore, a number of research works have been implemented on the solar irradiation prediction in recent years, in which a short survey is provided in the rest.

In [12], a review on the solar irradiance prediction models is provided. It is noted that solar irradiance is the most critical parameter due to its dependence on the climate situation and, thus, encompassing the most relevant frequencies originating from the weather condition. In [13], a data-driven method based on the wavelet recurrent neural network (RNN) is proposed for solar irradiance prediction for the next two-day time horizon. The wavelet transform helps to extract out the effective environmental features, such as wind speed, humidity, or temperature. The meteorological data provided at the University of Catania, Italy show the appropriate performance of the model. In [14], a probabilistic model based on joint probability distribution function is proposed to predict the solar irradiance. The proposed model uses the multidimensional kernel density assessment and Bayes rule to construct the probability functions. An estimation method is developed in [15] to evaluate the photovoltaic (PV) output power using the solar radiations. To make an economic system with reliable output power generation, the model is trained based on the practical data in different weather conditions such as the cloudy/rainy days. Lorenz *et al.* [16] presented a model for predicting the PV output power for the next three days on the weather forecasts of the European Centre for Medium-Range Weather Forecasts. The simulation results show that while the irradiance predictions for a single region show a root mean square error (RMSE) value of 36%, it can be improved up to 13% for the entire Germany. In [17], a combined physical model is suggested to predict the solar irradiance on inclined

Manuscript received October 28, 2020; revised May 8, 2021; accepted June 14, 2021. Date of publication July 12, 2021; date of current version December 17, 2021. This article was recommended by Associate Editor S. Bandyopadhyay. (*Corresponding author: Seyed Mohammad Jafar Jalali.*)

Seyed Mohammad Jafar Jalali, Abbas Khosravi, and Saeid Nahavandi are with the Institute for Intelligent Systems Research and Innovation, Deakin University, Waurn Ponds, VIC 3216, Australia (e-mail: sjalali@deakin.edu.au).

Sajad Ahmadian is with the Faculty of Information Technology, Kermanshah University of Technology, Kermanshah 67146, Iran.

Abdollah Kavousi-Fard is with the Department of Electrical and Electronics Engineering, Shiraz University of Technology, Shiraz 13876, Iran.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TSMC.2021.3093519>.

Digital Object Identifier 10.1109/TSMC.2021.3093519

surfaces. The proposed method is constructed by combining three decomposition models and four transposition models. An effective method is proposed in [18] to overcome the limitations of long short-term memory (LSTM) based on time-series encoding. Their model uses convolution operators to enrich the irradiance prediction for a limited dataset. Alsharif and Younes [19] proposed an adaptive neuro fuzzy prediction model for solar irradiance using the chaotic time-series dataset. The model assesses the changes in the solar radiation of Seoul city over a long time to help providing more accurate forecast results. An ensemble learning-based model is proposed in [20] to predict the solar irradiance based on the recorded real dataset accurately. Four models are deployed for reducing the forecast error variance with low bias.

Recently, different well-known deep learning techniques, including pretrained convolutional neural network (CNN) architectures, such as ResNet, VGG, and Alexnet, as well as the unsupervised generative adversarial network (GAN) models for the applications related to the solar irradiance forecasting, have been introduced. In [21], an accurate hybrid deep learning framework combining ResNet and RNN is proposed to forecast the solar irradiance changes in four different seasons. Jiang *et al.* [22] proposed a novel regression framework based on the AlexNet and VGG architectures to capture the global horizontal irradiance (GHI) accurately. In another work by Wang *et al.* [23], a data-driven deep learning model is proposed based on GAN and CNN models. They used a GAN model to increase the samples of training datasets and the CNN algorithm for solar irradiance prediction. In [24], an effective day-ahead PV power prediction framework is developed on the basis of long-short-term memory RNN (LSTM-RNN) to provide accurate daily pattern prediction information of particular days. Husein and Chung [25] applied an accurate day-ahead solar irradiance prediction model based on the deep learning theory (LSTM-RNN) for case studies in Germany, USA, Switzerland, and South Korea. In another study [26], solar irradiance is being efficiently forecasted for time horizons ranging from 1 min to 10 min ahead through utilizing a modified surface irradiance forecasting method with derived data for an actual sky image. Furthermore, Zhen *et al.* [27] proposed a computational mapping framework centered on deep learning applicable to solar PV power prediction in order to effectively accomplish the real-time mapping interaction between sky image and surface irradiance. After the preprocessing stage, the sky image datasets are clustered using the feature extraction of a convolutional autoencoder (AE) and the K -means clustering method. In the second stage, for surface irradiance, a hybrid mapping method relying on deep learning techniques is developed. The findings show that the model presented in this article is precise and can be stable compared to other benchmark deep learning models.

As it can be inferred from the above explanations, the solar irradiance prediction has been at the center of attention of researchers in recent years. This is mainly because the solar irradiance as the primary energy source for PV units contains nonlinear and hidden frequencies of the environment in its nature. This means that irradiance value fluctuates

according to the changes in the humidity, temperature, time of the day, etc. On the other hand, the design procedure of deep neural networks is always iterative and arduous, requiring technical expertise and practical experience in terms of trials and errors. Moreover, the tuning of the deep neural network hyperparameters is very critical since the efficiency of the network performance is closely linked to its architecture [28]–[30]. By reviewing previous studies in the field of solar irradiance forecasting, it can be concluded that most of them have designed the deep architectures manually using a trial-and-error process. This process is very heavy in terms of computational complexity since it takes a long time to execute a manually tuned architecture, which may not have the best predictive results [31]–[33]. Therefore, this article proposes a novel evolving deep learning-based model to produce accurate predictions of solar irradiance automatically without the need of manually tuning the deep learning architectures. To this end, a combinatorial model based on the optimized architecture of deep CNN and LSTM (CLSTM) is proposed to construct a highly reliable and accurate model.

In this article, due to the high complexity and nonlinearity in the GHI dataset, a new optimization method based on the sine-cosine algorithm (SCA) [34] is proposed to adjust the hyperparameter values of CNN-LSTM architectures. To be specific in the proposed method, we propose an improved version of the sine-cosine algorithm as an optimization approach to obtain the optimal values of the hyperparameters of the used deep CNN-LSTM neural networks. It should be noted that by combining the CLSTM models together, the proposed method can achieve both advantages of the CNN as a feedforward neural network and the LSTM as an RNN. It is a crucial task to enhance the performance of the deep neural networks by automatically finding the optimal values of their hyperparameters. Also, it is a challenging and time-consuming task to find optimal values of such hyperparameters. The proposed method provides an efficient approach to automatically determine optimal hyperparameter values leading to improve the accuracy of predictions. Moreover, we have modified the basic version of the sine-cosine algorithm by implementing a three-phase modification strategy with evolutionary operators, including opposition-based learning (OBL), Levy flight, and chaotic maps to improve its exploration and exploitation phases, increase the diversity of the population, and consequently premature convergence avoidance in the optimization process. Therefore, one of the main contributions of the proposed method is to develop an improved sine-cosine algorithm. The results of the experiments can prove such claims as the proposed method (i.e., MSCA-CLSTM method) achieves better performance than the model, which combines CLSTM without considering the modified sine-cosine algorithm (i.e., CLSTM method).

In brief, three major types can be outlined in the principal contributions of this work.

- 1) An accurate GHI solar irradiance forecasting method concentrating on SCA is introduced to configure the hyperparameters of deep CLSTM neural networks with a higher rate of convergence.

- 2) A three-phase modification is considered to boost the search capacity of SCA and avoid local optima trapping.
- 3) The performance of the proposed framework is assessed using three datasets gathered by the National Renewable Energy Laboratory from three different locations in the eastern states of the USA in the vicinity of Columbus, Detroit, and San Antonio. The experimental results show that the proposed method significantly outperforms other forecasting models.

The remainder of this article is organized as follows: Section II explains the proposed evolutionary deep learning model. The data used in the experiments and simulation results are discussed in Section III. Finally, the conclusion is provided in Section IV.

II. PROPOSED EVOLUTIONARY-BASED DEEP LEARNING MODEL

This section explains the proposed deep neuroevolution strategy to obtain the optimal architectures of the hybrid deep CNN-LSTM neural network.

A. Hyperparameters in deep CNN-LSTM

Many hyperparameters could be considered in the standard architecture of a CLSTM. In this research, we consider networks whose layers are sequentially stacked one after the other.

CNNs are regarded as one of the types of deep neural networks commonly utilized in numerous real-world applications. CNNs utilize three mapping layers, including the convolutional layer, the pooling layer, and the fully connected (i.e., dense) layer to process and represent data.

Convolutional layers are employed to identify input local connections, whereas the pooling layer slowly decreases the calculation corresponding to the target variable. Increasing the **number of convolution layers (N_c)** can have many kernels that are used to produce the same number of invariant feature maps for data from previous layers. Whilst the **degree of information extraction is deeper**, the **feature expression with a scale range is more precise**, since the number of kernels in each convolutional layer $kN_{i,i \in N_c}$ and the size of the kernels $kS_{i,i \in N_c}$ are important to the detail of the extracting information.

The pooling layer taking place after the convolutional layer is responsible for minimizing the size of the input by taking downsampling, which is defined by $pS_{i,i \in N_c}$. Once the feature maps have been obtained, they can be applied to a fully connected network. The fully connected layer determines a target dependent on input variables features. The **number of dense layers N_d** , the **number of neurons for each layer $nN_{i,i \in N_d}$** , and the **connectivity pattern of each dense layer $P_{i,i \in N_d}$** , are the **most important hyperparameters of fully connected layers**. A regularization concept $R_{i,i \in N_d}$ which implies either L1 (lasso) or L2 (ridge regression) and a dropout rate technique R_{dropout} , which consists of removing the proportional random collection of connections during training are implemented to control the overfitting weights of each layer.

Apart from the hyperparameters relevant to the CNN hierarchical architecture, there are some **basic hyperparameters**

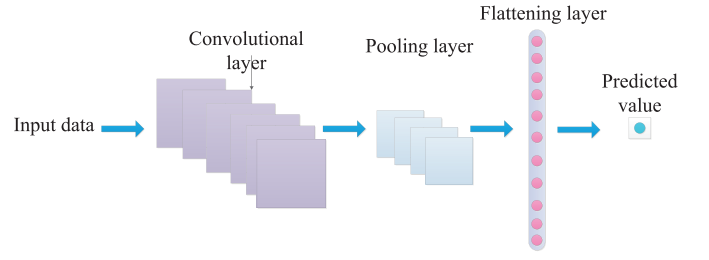


Fig. 1. General architecture of a CNN deep neural network.

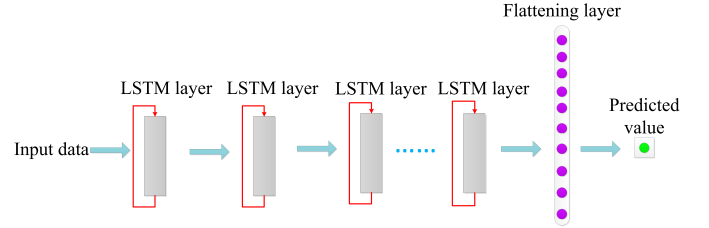


Fig. 2. General architecture of an LSTM deep neural network.

as **batch size S_{batch}** , and **learning rate L_{rate}** . A more optimal batch size can focus on providing better convergence and more effective gradient computing compared to full-batch gradient descent. For any training epoch, the learning rate determines how much the weights are adjusted. In order to simplify the modeling procedures, we apply an one-dimensional (1-D) convolution operator to directly forecast 1-D GHI datasets. In Fig. 1, a general representation of the CNN's architecture is shown.

There are units defined in LSTM models as input, output, and forget gates in memory blocks. Those give LSTMs the potential to update and control the flow of information in distinct blocks. This potential of LSTMs can be a competitive advantage in the case of a short-term solar-powered electricity forecast model because it would possibly allow for a grid system to continually update the next forecast using its input, output, and gate information on memory blocks. There are generally **five key hyperparameters for training the deep LSTM neural network**, including the **number of LSTM layers N_l** , **maximum epoch Max_e** , **neural units in each layer $uN_{i,i \in N_l}$** , and **batch size IS_{batch}** , having been applied in this work. A general figure from an LSTM architecture is illustrated in Fig. 2.

This study designs a CLSTM hybrid model (convolutional-LSTM) in which CNN layers are deployed to extract features related to potential solar GHI changes and the LSTM layer is utilized to integrate these features in the low-latency prediction of GHI data in time series for developing a dynamic feature extraction and predictive model with deeper learning algorithms. Fig. 3 demonstrates the topological architecture of the GHI prediction based on the CLSTM hybrid framework.

B. Modified Sine-Cosine Algorithm

SCA is an intelligent evolutionary approach oriented on the sine and cosine functions. For exploration and exploitation phases, SCA uses two position update equations, which are

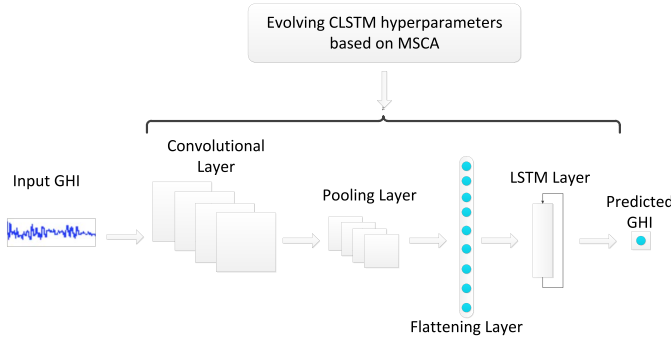


Fig. 3. Process of the development of the MSCA-CLSTM.

TABLE I
LIST OF EXPRESSIONS AND VALUATIONS OF CLSTM
HYPERPARAMETERS

Expression	Valuation List
$N_c = \psi$	$N_c(\psi)_{\psi \in [1,8]} \in [1, 2, 3, \dots]$
$fN_{1 < i < N_c} = 2^{\psi+2}$	$fN(\psi)_{\psi \in [1,6]} \in [8, 16, 32, \dots]$
$kS_{1 < i < N_c} = (2 \times \psi) + 1$	$kS(\psi)_{\psi \in [1,5]} \in [3, 5, 7, 9, 11]$
$N_p = \chi$	$N_p(\chi)_{\chi \in [1,3]} \in [1, 2, 3]$
$s_{1 < i \leq N_p} = \overline{p(\psi)}$	$nN(\chi)_{\chi \in [1,10]} \in [1, 2, 3, \dots]$
$R_{\text{dropout}} = (\psi + 3) \times 0.05$	$R_{\text{dropout}}(\psi)_{\psi \in [1,8]} \in [0.2, 0.25, \dots]$
$S_{\text{batch}} = 10 \times \psi$	$S_{\text{batch}}(\gamma)_{\gamma \in [1,10]} \in [10, 20, \dots]$
$L_{\text{rate}} = 0.001 + 0.005 \times (\psi - 1)$	$L_{\text{rate}}(\psi)_{\psi \in [1,21]} \in [0.001, 0.006, \dots]$
$N_l = \eta$	$N_l(\eta)_{\eta \in [1,3]} \in [1, 2, 3]$
Max_e	$Max_e(\eta)_{\eta \in [1,500]} \in [1, 2, \dots]$
$uN_{i,i \in N_l}$	$N_c(\psi)_{\psi \in [1,300]} \in [1, 2, 3, \dots]$
$lS_{\text{batch}} = 10 \times \eta$	$lS_{\text{batch}}(\eta)_{\eta \in [1,3]} \in [10, 20, 30]$

described as follows:

$$x_i^{k+1} = x_i^k + r_1 \cdot \sin(r_2) \cdot |r_3 p_i^k - x_i^k| \quad (1)$$

$$x_i^{k+1} = x_i^k + r_1 \cdot \cos(r_2) \cdot |r_3 p_i^k - x_i^k| \quad (2)$$

where x_i^k represents the current solution position, r_1 , r_2 , and r_3 are three random numbers that are randomly distributed between 0 and 1, and p_i^k specifies the target point position. Such two updated rules are combined as follows:

$$x_i^{k+1} = \begin{cases} x_i^k + r_1 \cdot \sin(r_2) \cdot |r_3 p_i^k - x_i^k| & r_4 < 0.5 \\ x_i^k + r_1 \cdot \cos(r_2) \cdot |r_3 p_i^k - x_i^k| & r_4 \geq 0.5 \end{cases} \quad (3)$$

where r_4 denotes to a random number in the interval [0, 1].

There are four control parameters in the SCA. In particular, r_1 is used to define the next search area, r_2 is used to describe motion direction and how far it will go, r_3 handles the effect on the current movement of the destination, and r_4 flips between the components of sine and cosine. In addition, r_1 , r_2 , and r_3 are updated in each iteration to boost the optimization performance. The random number r_1 is adapted in order to find a balance between exploration and exploitation during the searching procedure using the following equation:

$$r_1 = a - k \cdot \frac{a}{k_{\max}} \quad (4)$$

where a is a constant number, set to 2, k represents the current iteration, and $k_{\max}$ denotes to the maximum iteration number.

The SCA has unique capabilities, making it a robust optimizer for nonconvex and nonlinear constraint optimization

problems. Its key benefits are few tuning parameters, ease of deployment, and efficient global search mechanism. However, its performance can be enhanced even further by equipping this algorithm with strong search strategies. Thus, we introduce here a modification method of three phases to boost the efficiency of the SCA. In the following, each of these three phases is described.

Modification Phase 1: This modification phase employs the OBL strategy to establish the diversification of the population. Assume that $x \in [a, b]$ represents a real number, and the opposition number of x is defined as x' , which is represented as

$$x' = a + b - x \quad (5)$$

where a and b denote to the lower and upper bounds. The definition above can be generalized to higher dimensions. Let $x_i = (x_{i1}, x_{i2}, \dots, x_{ij})$, where $x_{ij} \in [a_j, b_j]$ and $j = 1, 2, \dots, n$. For a candidate solution x_i , the opposite vector of x_i is defined as

$$x'_i = (x'_{i1}, x'_{i2}, \dots, x'_{ij}) \quad (6)$$

where $x'_{ij} = a_j + b_j - x_{ij}$. The opposite position x'_i is replaced by the corresponding solution x_i on the basis of the fitness function during the optimization process. If $f(x_i)$ is considerably better than $f(x'_i)$, then x_i is not adjusted; otherwise, $x_i = x'_i$.

Modification Phase 2: This modification phase utilizes the Levy flight algorithm to maximize the efficiency of resource searches and prevent premature convergence using the following equation:

$$X_i^{\text{new}} = X_i + \infty \oplus \text{Levy}(\beta) \quad (7)$$

where ∞ represents a parameter for the step size, which is defined as

$$\infty \oplus \text{Levy}(\beta) \sim 0.01 \frac{p}{|q|^{1/\beta}} (X_i^k - X_{\text{best}}^k) \quad (8)$$

where p and q are obtained using the following equations:

$$p \sim N(0, \phi_u^2), q \sim N(0, \phi_v^2) \quad (9)$$

$$\phi_u = \left[\frac{\Gamma(1+\beta) \times \sin(\pi \times \beta/2)}{\Gamma[(1+\beta)/2] \times \beta} \right]^{1/\beta}, \phi_v = 1 \quad (10)$$

where the standard gamma function is symbolized by Γ while its range is in [0, 2].

Modification Phase 3: In this phase, chaotic maps are used to modify the key parameters of the SCA in order to improve its convergence speed. In this study, four chaotic maps including the sawtooth map (11), sine map (12), tent map (13), and Chebyshev map (14) are applied to tune r_1 , r_2 , r_3 , and r_4 parameters of the SCA, respectively. The formulas for these chaotic maps are as follows:

$$r_1 = 2r_1 \bmod 1 \quad (11)$$

$$r_2 = 0.25a \sin(\pi r_2), \quad 0 < a \leq 4 \quad (12)$$

$$r_3 = \begin{cases} \frac{r_3}{10}, & r_3 < 0.7 \\ \frac{10}{3}(1 - r_3), & r_3 \geq 0.7 \end{cases} \quad (13)$$

$$r_4 = \cos(4 \cos^{-1}(r_4)). \quad (14)$$

Fig. 4 illustrates the proposed SCA flowchart.

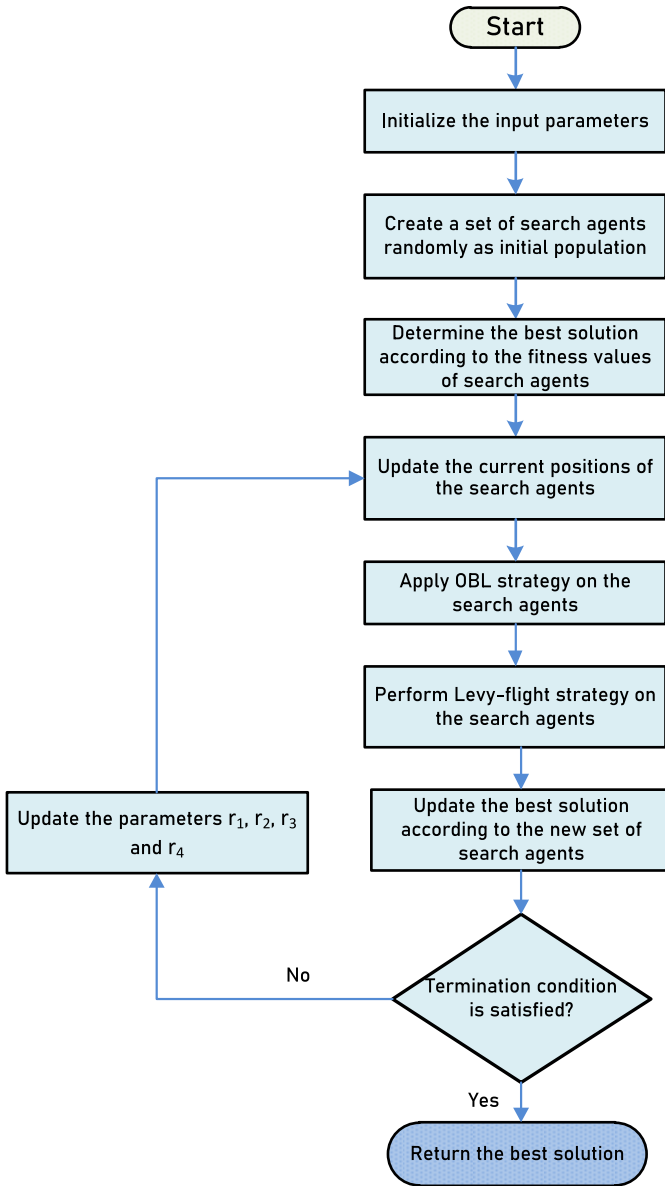


Fig. 4. Flowchart of the proposed SCA strategy.

C. Proposed MSCA-CLSTM Method

In this section, we describe the proposed GHI prediction technique named MSCA-CLSTM. This approach is aimed at using the improved SCA for optimizing deep CLSTM neural network hyperparameters to improve forecasting accuracy for the solar GHI datasets. Two major issues, including solution representation and the fitness function calculation, should be addressed before deploying MSCA. It is worth noting that the proposed approach optimizes all CLSTM hyperparameters described in the previous section. Each solution in MSCA can therefore be defined as a vector with 12 dimensions, each of which aligns to one of the 12 hyperparameters. For instance, the learning rate or dropout rate are the continuous-value hyperparameters which MSCA obtains their optimum values. The batch size, number of CNN or LSTM layers, maximum epoch, and neural units, in comparison, are other hyperparameters with discrete values. Since MSCA continuously explores

the solution space, we need to translate the achieved optimum values to their respective discrete hyperparameters. As a result, we design the following equation to convert each real value to an integer value

$$y_{ij} = \left\lfloor g_j * \frac{x_{ij} - lb}{ub - lb} + 0.5 \right\rfloor, \quad j = 1, \dots, n \quad (15)$$

where g_j denotes to the total number of the item type j , x_{ij} indicates the real number corresponds to the j th dimension of the solution X_i , y_{ij} is the integer value converted, lb and ub represent the lower and upper bounds of the search space, respectively.

In the proposed MSCA-CLSTM algorithm, a population with n individuals is generated, where each solution is represented as a 12-D vector X_{ij} , $i = 1, \dots, n$ and $j = 1, \dots, 12$ and each dimension j is equal to one of the 12 CLSTM hyperparameters. After initializing the population and chaotic maps, new solutions can be reached by continuously updating current positions of the solutions using (3). In addition, the OBL and Levy flight strategies are used for the updated positions in order to balance between the exploration and exploitation by use of (6) and (7). The procedure continues until the termination condition is met and the best solution obtained is known as the final result. This solution can be considered as the CLSTM hyperparameter optimal values. In order to assess the effectiveness of each solution, a fitness function needs to be specified. For this reason, the data from the input time series are split in two sets of training and testing. The training set is needed to optimize the CLSTM hyperparameters with the aid of MSCA, whereas the test set evaluates the output of the final GHI prediction model. Assume that $\vec{y}$ is a vector that can represent the actual data from the GHI time series for M time steps as follows:

$$\vec{y} = (y(0), y(1), \dots, y(M-1)) \quad (16)$$

where the actual GHI value for the time step t is denoted by $y(t)$. The proposed GHI prediction model is intended to forecast GHI values from the next N time steps using the CLSTM neural network, as follows:

$$\vec{\hat{y}} = (\hat{y}(M), \hat{y}(M+1), \dots, \hat{y}(M+N-1)) \quad (17)$$

where $\vec{\hat{y}}(t)$ represents the forecast GHI value for the time step t . It is necessary to notice that any MSCA solution is required to initialize a CLSTM model based on the obtained values of hyperparameters. Thus, a fitness function can be taken into consideration for the efficiency of the configured CLSTM model on prediction of GHI values. In order to achieve this, the CLSTM's input vectors are presented based on the training data using (16). The CLSTM model is then used to forecast the GHI values of the next N time-steps that are indicated with (17). The mean square error can be used to measure the fitness value of each MSCA solution as follows:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - y'_i)^2 \quad (18)$$

where y_i denotes to the actual GHI value and y'_i represents the predicted GHI value achieved by the deep CLSTM neural

TABLE II
PARAMETERS AND FLOPS OF DIFFERENT BENCHMARK ALGORITHMS

Model	Parameters and flops								
	Activation function	Batch size	Larning rate	No. iterations	Epochs	No. filters	Pooling size	Dropout rate	Population size
Auto-LSTM	ReLU	1, 0, 050	0.001	-	50	32	2	0.2	-
XGBF-DNN	ReLU	32	0.001	-	150	64	-	0.2	-
LSTM	ReLU	50	0.001	-	50	32	2	0.2	-
CLSTM	ReLU	512	0.001	-	200	32	2	0.3	-
DWT-CLSTM	ReLU	24	0.001	-	200	64	3	0.2	-
MEA-ANN	-	-	-	20	-	-	-	-	200
SCA-CLSTM	ReLU	[10-100]	[0.001-0.101]	20	[1-500]	[8-256]	[1-10]	[0.2-0.55]	30
MSCA-CLSTM	ReLU	[10-100]	[0.001-0.101]	20	[1-500]	[8-256]	[1-10]	[0.2-0.55]	30

Algorithm 1 Pseudocode of the Proposed GHI Forecasting Model (MSCA-CLSTM)

```

1: Input: pop_size (population size) and n (maximum number of iterations).
2: Output: Predicted solar GHI irradiance.
3: Begin algorithm:
4: Split dataset into two sets including training set Tr and test set Te;
5: Initialize the search agent population  $X_i$  ( $i=1,2,\dots, pop\_size$ );
6: Initialize parameter  $r_1, r_2, r_3$  and  $r_4$ ;
7: for (each solution  $X_i$  in the population) do
8:   Set a CLSM model based on the values of solution  $X_i$  as the
   hyperparameters;
9:   Calculate the fitness of solution  $X_i$  using Eq. (18) as the MSE error
   of CLSTM model according to the training set Tr;
10: end for
11: Let B the best search agent based on the calculated fitness values;
12: while (number of iterations < n) do
13:   for each solution  $X_i$  in the population do
14:     Update the current position of  $X_i$  using Eq. (3);
15:     Apply the OBL mechanism based on Eq. (6)
16:     Apply the levy flight strategy using Eq. (7);
17:     Set a CLSTM model based on the values of solution  $X_i$  as the
     hyperparameters;
18:     Calculate the fitness of solution  $X_i$  using Eq. (18) as the MSE
     error of CNN model obtained based on the training set Tr;
19:     if the fitness of solution  $X_i$  is better than the fitness of B then
20:       Set  $B=X_i$ ;
21:     end if
22:   end for
23:   Update  $r_1, r_2, r_3$  and  $r_4$  according to Eqs. (11-14), respectively;
24:   Increase the number of iterations by 1;
25: end while
26: Set a CLSTM model based on the values of hyperparameters obtained
   by the best solution B;
27: Predict the solar GHI data in the test set Te using the CLSTM model;
28: End algorithm

```

network. Evidently, a small *MSE* value for each solution has a superior fitness value, and vice versa. Thus, the purpose of the proposed method is to obtain a solution with the lowest *MSE* value containing the optimal values of CLSTM hyperparameters. This results in the achievement of a CLSTM model with the highest performance in the test set for GHI forecasting. The overall details of the proposed MSCA-CLSTM technique are represented by Algorithm 1.

III. EXPERIMENTAL RESULTS AND DISCUSSIONS

This section represents the experiments performed on three well-known datasets to compare the performance of the proposed MSCA-CLSTM method with other models. To this end, the proposed method is compared with different algorithms used in solar energy forecasting applications. These benchmark algorithms are listed as follows.

TABLE III
LIST OF THE PRIMARY CLSTM ARCHITECTURE HYPERPARAMETERS

Hyperparameters	Symbol
Number of convolutional layers	N_c
Number of filters in each convolutional layer	$fN_{i,i \in N_c}$
Kernel size in each convolutional layer	$kS_{i,i \in N_c}$
Number of pooling layers	N_p
Pooling size after each convolutional layer	$pS_{i,i \in N_c}$
Dropout	$R_{dropout}$
Batch size	S_{batch}
Learning rate	L_{rate}
Number of LSTM layers	N_l
Maximum epoch	Max_e
Neural unites in each LSTM layer	$uN_{i,i \in N_l}$
LSTM batch size	lS_{batch}

- 1) **DWT-CLSTM** [35]: It is an improved hybrid deep learning algorithm focused on three concepts, including wavelet decomposition (WD), deep CNN and deep LSTM models.
- 2) **MEA-ANN** [36]: It is a powerful hybrid algorithm based on the mind evolutionary algorithm (MEA) and artificial neural network (ANN). In the MEA-ANN model, the MEA is employed to choose the optimal initial weights and biases for the artificial neural networks.
- 3) **Auto-LSTM** [37]: This hybrid model incorporates AE's feature learning through a temporal use of an LSTM context using the AE to carry out feature learning, and the LSTM network is connected to the encoding portion of the AE.
- 4) **XGBF-DNN** [38]: This ensemble model is based on the concepts of extreme gradient boosting forest and deep neural networks in which, for avoiding the overfitting issue, the base models are combined by ridge regression technique.
- 5) **LSTM** [39]: In this work, the authors used the deep LSTM network for predicting the solar irradiance. The experimental results reveal that the proposed LSTM model outperforms other competitive algorithms for single output forecasting.
- 6) **CLSTM** [40]: In this technique, the authors have utilized the combination of two deep neural network models including the CLSTM that provide an excellent alternative for short-term solar radiation forecasting.
- 7) **SCA-CLSTM**: This model is based on the original version of SCA hybridized with deep CLSTM. This powerful model can show the search ability of the proposed MSCA. The hyperparameters shown in Table I have

been evolved by the SCA model in order to optimize the deep CLSTM architecture efficiently.

Three evaluation metrics, including RMSE, the Pearson correlation coefficient (r), and mean absolute error (MAE) are used to verify the performance of the compared algorithms. These metrics are calculated as follows:

$$\text{RMSE} = \sqrt{\left(\frac{1}{n}\right) \sum_{i=1}^n (y'_i - y_i)^2} \quad (19)$$

$$R = \frac{\sum_{i=1}^n (y'_i - \bar{y}') (y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (y'_i - \bar{y}')^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (20)$$

$$\text{MAE} = \left(\frac{1}{n}\right) \sum_{i=1}^n |y'_i - y_i|. \quad (21)$$

where y'_i reflects the predicted values of actual (y_i) GHI values and n denotes to the number of data points in the test set. It should be mentioned that the higher r value and the lower RMSE and MAE values, the more accurate the model predicts. The proposed MSCA-CLSTM and other compared algorithms are implemented in Keras, a high-level library of TensorFlow back-end under the version 3.6 of Python language programming, CUDA version of 10.1 and executed on a machine configuring with NVIDIA GTX 1080Ti GPU, 32 GB of RAM, and Intel Core i7 CPU.

A. Data and Parameter Setting

We utilize the National Renewable Energy Laboratory datasets for solar irradiance time series gathered from three solar locations in the eastern states of the USA in the vicinity of Columbus, Detroit, and San Antonio [41]. The 8760 data points on each site comprises the GHI time series for the whole year 2018 with 1-h intervals. 75% of each dataset is allocated to training set and the remaining is considered for test set. For all datasets, the GHI grows between 8 A.M. and 1 P.M. and then drops to zero between 6 A.M. and 8 P.M. Similar to [42], we use mutual information (MI) and choose timestamps with MI values above the threshold $\tau = 0.4$ in order to select input features. Using this approach, the future GHI values are forecast by 53 steps. We test each algorithm for ten independent runs and the average is reported as the final result. The CLSTM hyperparameters are optimized using the MSCA with the initial population size of 30, the termination criterion of 20 iterations, and ten independent runs. The initialized parameters and flops for the proposed algorithm and the other benchmark algorithms are reported in Table II. The configurations for parameters of the compared benchmark algorithms are chosen based on the best values reported in their corresponding references. Moreover, for the hyperparameters that were not mentioned their corresponding values in their referenced works, we selected the best hyperparameter values based on a greedy search strategy. For our proposed MSCA-CLSTM model, the values and expressions for all hyperparameters evolved in the evolutionary procedure have been reported in Tables III and I as we mentioned before.

B. Performance Comparison

Comprehensive experiments are conducted to compare the performance of the proposed MSCA-CLSTM method with other models based on three evaluation metrics, including RMSE, MAE, and Pearson. Also, to make a better analysis of the results, different forecasting horizons are considered in a range of 1-step to 6-step. Table IV reports the results of GHI time-series forecasting on the Columbus dataset. These results reveal that the proposed method outperforms other forecasting models in terms of three evaluation metrics and also different forecasting horizons. For instance, the proposed method obtains the best results for 1-step time-series forecasting with the values of 0.0414, 0.9590, and 0.0229 for RMSE, Pearson, and MAE metrics, respectively. At the same time, the SCA-CLSTM model places at the second best position by obtaining the values of 0.0463, 0.9508, and 0.0277 for RMSE, Pearson, and MAE metrics, respectively. As we can see from these results, the RMSE and MAE increase and Pearson values decrease for all compared models, respectively, when the forecasting horizon is changed from 1-step to 6-step. Therefore, it can be concluded that increasing the forecasting horizon has a negative effect on the performance of the forecasting models. The experimental results on the Detroit dataset are shown in Table V. It can be seen from these results that the proposed method is the best performer in comparison to other models based on MAE, RMSE, and Pearson metrics and also different forecasting horizons. For instance, RMSE, Pearson, and MAE values of the 1-step forecasting horizon obtained by the proposed method are 0.0413, 0.9480, and 0.0229, respectively. The second best performer based on RMSE, Pearson, and MAE metrics is SCA-CLSTM which its corresponding results are 0.0454, 0.9406, and 0.0277, respectively. Looking more closely at these results, we find that as the time step increases from one to six, the amount of error predicted by most models increases which means more difficult to predict GHI data points. Regarding the results of the San Antonio dataset reported in Table VI, among the compared models, the MSCA-CLSTM model is the pioneer model in all six different time steps. For instance, with 1-step horizon forecasting, the RMSE, Pearson, and MAE metrics with values of 0.0524, 0.9637, and 0.0312 are the best values among all the compared models. For other time steps, as the time steps increase, the values of RMSE and MAE increase and the Pearson values decrease.

In summary, we have compared the proposed method which is based on the combination of CLSTM models with the models based on only CNN or LSTM. The experimental results reported in Tables IV–VI demonstrate that the proposed combined model significantly outperforms other CNN-based and LSTM-based models. Therefore, it can be concluded that the combination of the CLSTM models leads to obtain better performance in comparison to the models, which are based on only one of the CNN or LSTM models. It should be noted that by combining the CLSTM models together, the proposed method can obtain both advantages of the CNN as a feedforward neural network and the LSTM as an RNN. On the other hand, the results of the experiments prove that the proposed

TABLE IV
PERFORMANCE COMPARISON OF DIFFERENT FORECASTING MODELS BASED ON THE COLUMBUS DATASET

Method	Time Step																	
	1-Step			2-Step			3-Step			4-Step			5-Step			6-Step		
	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE
Auto-LSTM	0.0559	0.9257	0.0319	0.0771	0.8732	0.0451	0.0892	0.8257	0.0513	0.0993	0.7845	0.0573	0.1124	0.7468	0.0602	0.1131	0.7567	0.0647
XGBF-DNN	0.0551	0.9287	0.3148	0.0765	0.8786	0.0442	0.0889	0.8343	0.0507	0.0986	0.7856	0.0566	0.1098	0.7562	0.0631	0.1129	0.7465	0.0639
LSTM	0.0541	0.9315	0.0306	0.0756	0.8881	0.0431	0.0886	0.8278	0.0505	0.0983	0.7840	0.0563	0.1076	0.7604	0.0655	0.1105	0.7435	0.0621
CLSTM	0.0557	0.9428	0.0322	0.0724	0.8802	0.0408	0.0861	0.8272	0.0483	0.0966	0.7972	0.0556	0.1063	0.7570	0.0621	0.1082	0.7526	0.0633
DWT-CLSTM	0.0506	0.9414	0.0287	0.0708	0.8803	0.0395	0.0834	0.8403	0.0490	0.0963	0.7974	0.0554	0.1057	0.7586	0.0604	0.1095	0.7273	0.0626
MEA-ANN	0.0491	0.9540	0.0282	0.0711	0.8746	0.0408	0.0817	0.8429	0.0540	0.0876	0.8186	0.0550	0.0918	0.8237	0.0555	0.1061	0.7318	0.0705
SCA-CLSTM	0.0463	0.9508	0.0277	0.0658	0.9120	0.0399	0.0686	0.8890	0.0372	0.0775	0.8493	0.0451	0.0849	0.8226	0.0569	0.0943	0.8132	0.0538
MSCA-CLSTM	0.0414	0.9590	0.0229	0.0583	0.9204	0.0323	0.0659	0.8929	0.0359	0.0744	0.8606	0.0387	0.0802	0.8404	0.0433	0.0843	0.8184	0.0477

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT FORECASTING MODELS BASED ON THE DETROIT DATASET

Method	Time Step																	
	1-Step			2-Step			3-Step			4-Step			5-Step			6-Step		
	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE
Auto-LSTM	0.0534	0.9286	0.0324	0.0783	0.8340	0.0456	0.0941	0.7799	0.0520	0.1076	0.7038	0.0576	0.1135	0.6877	0.0605	0.1206	0.6725	0.0651
XGBF-DNN	0.0529	0.9298	0.0319	0.0779	0.8340	0.0452	0.0939	0.7801	0.0504	0.1073	0.7046	0.0573	0.1128	0.6931	0.0602	0.1194	0.6789	0.0648
LSTM	0.0553	0.9227	0.0306	0.0807	0.8334	0.0431	0.0940	0.7799	0.0505	0.1070	0.7047	0.0563	0.1114	0.6964	0.0655	0.1187	0.6717	0.0621
CLSTM	0.0550	0.9228	0.0322	0.0776	0.8413	0.0408	0.0927	0.7817	0.0483	0.1044	0.7217	0.0556	0.1099	0.6756	0.0621	0.1178	0.6790	0.0633
DWT-CLSTM	0.0549	0.9232	0.0287	0.0771	0.8419	0.0395	0.0903	0.7899	0.0490	0.1018	0.7307	0.0554	0.1034	0.7074	0.0604	0.1166	0.6633	0.0626
MEA-ANN	0.0484	0.9332	0.0282	0.0718	0.8709	0.0408	0.0895	0.7926	0.0540	0.1007	0.7274	0.0550	0.1005	0.6947	0.0555	0.1130	0.6576	0.0705
SCA-CLSTM	0.0454	0.9406	0.0277	0.0659	0.8880	0.0399	0.0740	0.8631	0.0372	0.0854	0.8246	0.0451	0.0912	0.7862	0.0569	0.0931	0.7949	0.0538
MSCA-CLSTM	0.0413	0.9480	0.0229	0.0590	0.9059	0.0323	0.0696	0.8596	0.0359	0.0804	0.7946	0.0387	0.0889	0.7648	0.0433	0.0855	0.7607	0.0477

TABLE VI
PERFORMANCE COMPARISON OF DIFFERENT FORECASTING MODELS BASED ON SAN ANTONIO DATASET

Method	Time Step																	
	1-Step			2-Step			3-Step			4-Step			5-Step			6-Step		
	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE
Auto-LSTM	0.0615	0.9496	0.0451	0.0813	0.9145	0.0578	0.0841	0.9135	0.0598	0.0891	0.9021	0.0641	0.0922	0.8783	0.0690	0.0962	0.8288	0.0746
XGBF-DNN	0.0612	0.9507	0.4483	0.0806	0.9116	0.0573	0.0840	0.9168	0.0587	0.0889	0.8985	0.0640	0.0921	0.8763	0.0665	0.0961	0.8238	0.0743
LSTM	0.0629	0.9482	0.0469	0.0832	0.9127	0.0582	0.0853	0.9129	0.0611	0.0900	0.8994	0.0652	0.0930	0.8760	0.0706	0.0960	0.8297	0.0749
CLSTM	0.0621	0.9456	0.0470	0.0802	0.9169	0.0573	0.0850	0.9131	0.0609	0.0899	0.9022	0.0650	0.0923	0.8770	0.0695	0.0968	0.8275	0.0741
DWT-CLSTM	0.0583	0.9537	0.0437	0.0750	0.9255	0.0519	0.0816	0.9146	0.0578	0.0870	0.9029	0.0628	0.0899	0.8804	0.0678	0.0949	0.8338	0.0725
MEA-ANN	0.0605	0.9513	0.0454	0.0772	0.9235	0.0545	0.0825	0.9134	0.0592	0.0875	0.9032	0.0640	0.0902	0.8811	0.0684	0.0951	0.8327	0.0729
SCA-CLSTM	0.0552	0.9584	0.0421	0.0735	0.9289	0.0493	0.0806	0.9164	0.0570	0.0868	0.9034	0.0622	0.0893	0.8826	0.0672	0.0946	0.8345	0.0719
MSCA-CLSTM	0.0524	0.9637	0.0312	0.0710	0.9315	0.0447	0.0783	0.9190	0.0539	0.0837	0.9069	0.0581	0.0877	0.8865	0.0648	0.0923	0.8396	0.0699

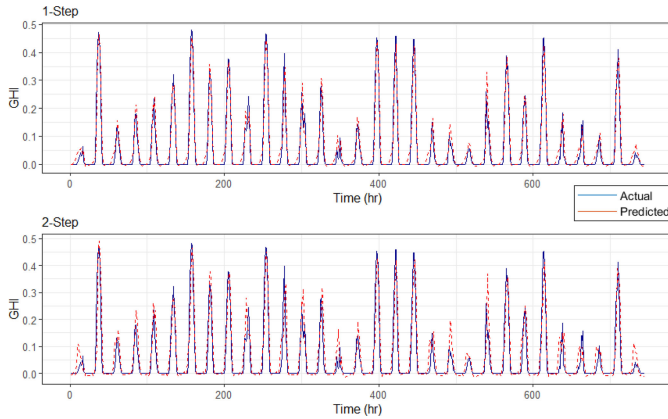


Fig. 5. GHI forecasting results of different time steps obtained by the proposed algorithm for Columbus—December.

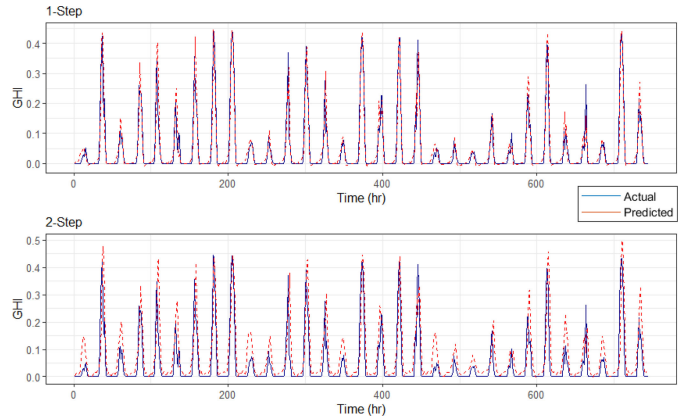


Fig. 6. GHI forecasting results of different time steps obtained by the proposed algorithm for Detroit—December.

sine-cosine optimization algorithm can improve the prediction accuracy of deep neural networks. This is due to the fact that the proposed method (i.e., MSCA-CLSTM) achieves better performance than the model which combines CLSTM without considering the modified sine-cosine algorithm (i.e., CLSTM).

Figs. 5–7 demonstrate the actual values (indicated with blue) of the GHI time series and their predicted values (indicated with red) obtained by the proposed method for different forecasting horizons. As we can see from these plots, the predicted

values are mainly close to the actual values showing that the proposed method can be adopted as a powerful forecasting model. Moreover, our novel algorithm follows sharp drops and correctly predicts extreme values. Furthermore, from the analysis of Figs. 5–7, our proposed model represents apparently a better curve fitting of the actual solar irradiance time series and smaller residual errors for 1-step and 2-step ahead forecasting horizons.

Figs. 8–10 show the convergence profile of the proposed MSCA-CLSTM method based on the Columbus, Detroit, and

TABLE VII
AVERAGE OF TIME CONSUMED IN SECONDS FOR DIFFERENT FORECASTING MODELS

Model	Optimization time			Training time			Testing time		
	Columbus	Detroit	San Antonio	Columbus	Detroit	San Antonio	Columbus	Detroit	San Antonio
Auto-LSTM	-	-	-	62	65	66	23	26	28
XGBF-DNN	-	-	-	61	63	69	25	27	29
LSTM	-	-	-	63	65	68	24	25	27
CLSTM	-	-	-	64	66	70	25	27	30
DWT-CLSTM	-	-	-	72	74	77	31	33	34
MEA-ANN	341	345	356	65	68	71	26	27	30
SCA-CLSTM	332	337	344	59	63	64	22	22	25
MSCA-CLSTM	317	325	329	58	61	63	19	20	22

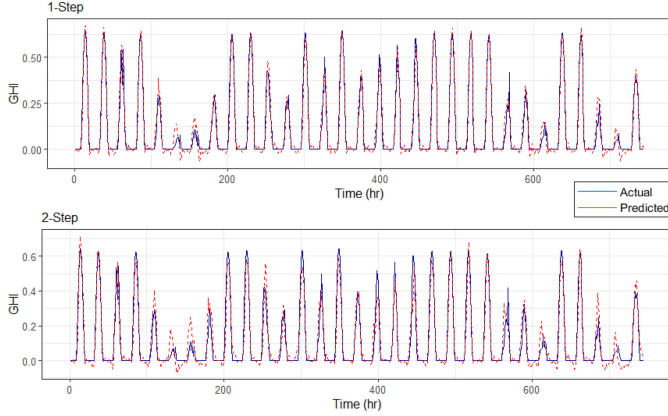


Fig. 7. GHI forecasting results of different time steps obtained by the proposed algorithm for San Antonio—December.

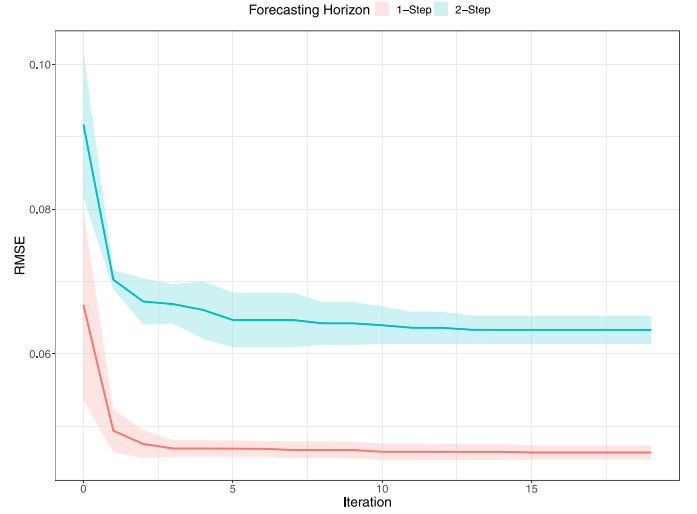


Fig. 9. Convergence profile of the proposed MSCA-CLSTM algorithm for two forecasting horizons of Detroit case study.

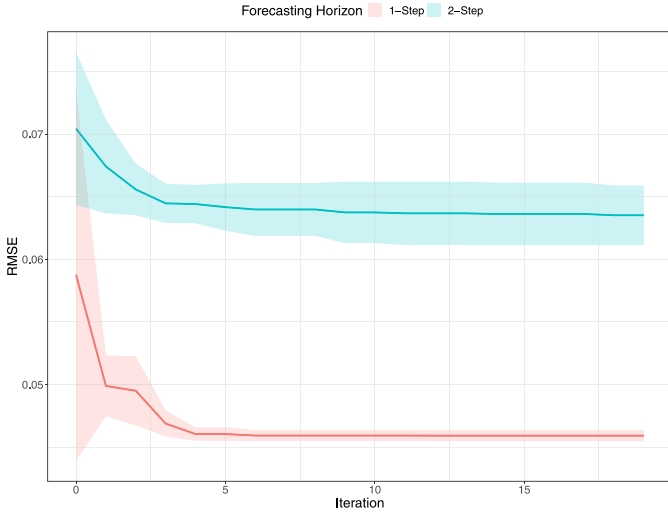


Fig. 8. Convergence profile of the proposed MSCA-CLSTM algorithm for two forecasting horizons of Columbus case study.

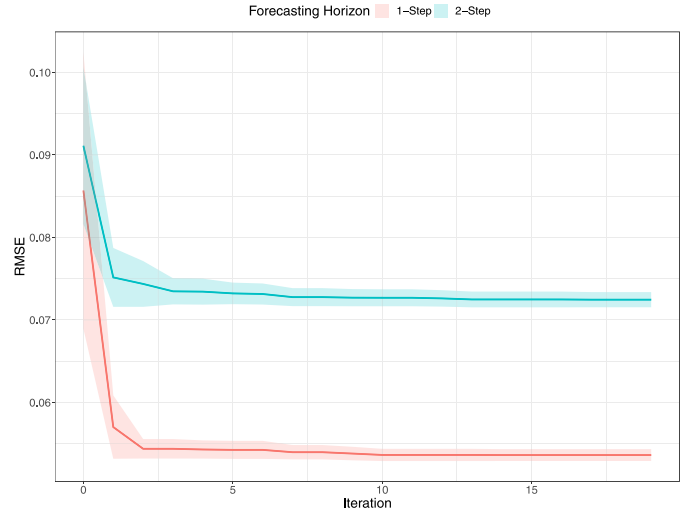


Fig. 10. Convergence profile of the proposed MSCA-CLSTM algorithm for two forecasting horizons of San Antonio case study.

San Antonio datasets, respectively. In these figures, we show the convergence profiles for 1-step and 2-step forecasting horizons for RMSE metric. These results reveal that the proposed method has a high convergence speed as converges after a few iterations for Columbus, Detroit, and San Antonio datasets. Therefore, it can be concluded that the proposed hyperparameters optimization model has a significant impact on the convergence speed improvement. Figs. 11–13 show the violin

plots of 12 hyperparameters used by the proposed algorithm for Columbus, Detroit and San Antonio case studies. The number of optimal parameters in thicker areas of the violin plots is more prevalent. For example, it is clear from the violin plots of the hyperparameter kernel size (k_s) for the Columbus

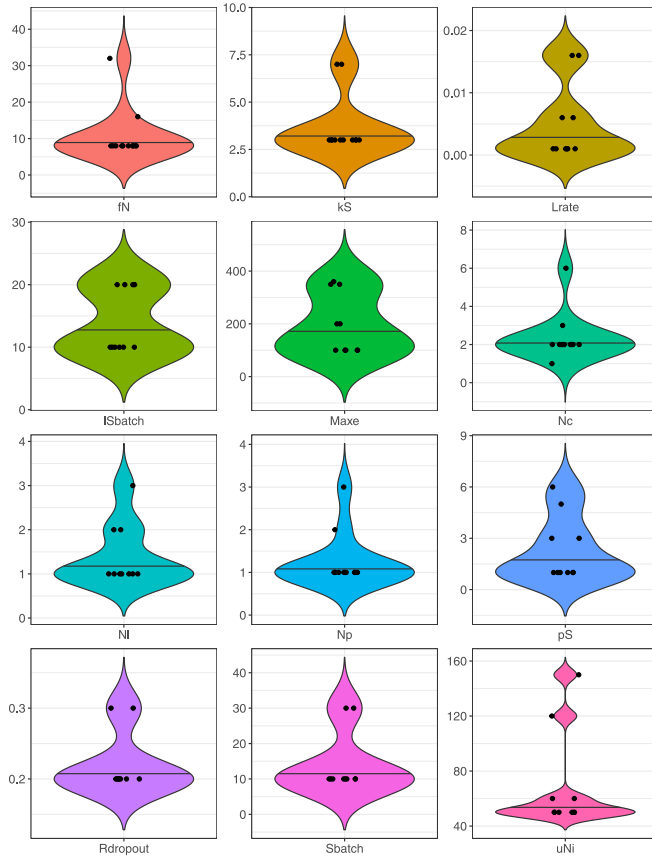


Fig. 11. Violin plots of the values obtained from the evolved hyperparameters applied in the MSCA-CLSTM algorithm for the Columbus case study of the 2-step ahead forecasting horizon.

TABLE VIII
PERFORMANCE METRICS OF DIFFERENT SEASONS
FOR THE COLUMBUS DATASET

Method	Time Step											
	Spring			Summer			Autumn			Winter		
	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE
Auto-LSTM	0.0341	0.9488	0.0228	0.0437	0.9396	0.0317	0.0242	0.9515	0.0192	0.0258	0.9550	0.0191
XGBF-DNN	0.0333	0.9518	0.3057	0.0429	0.9426	0.3146	0.0234	0.9546	0.3021	0.3087	0.9580	0.0183
LSTM	0.0323	0.9547	0.0215	0.0419	0.9454	0.0304	0.0224	0.9574	0.0179	0.0245	0.9608	0.0173
CLSTM	0.0339	0.9659	0.0231	0.0435	0.9567	0.0320	0.0240	0.9687	0.0195	0.0261	0.9721	0.0189
DWT-CLSTM	0.0288	0.9645	0.0196	0.0384	0.9553	0.0285	0.0189	0.9672	0.0160	0.0226	0.9707	0.0138
MEA-ANN	0.0259	0.9781	0.0189	0.0351	0.9690	0.0279	0.0160	0.9809	0.0155	0.0222	0.9845	0.0109
SCA-CLSTM	0.0231	0.9749	0.0184	0.0323	0.9658	0.0274	0.0132	0.9777	0.0150	0.0217	0.9812	0.0081
MSCA-CLSTM	0.0182	0.9832	0.0136	0.0274	0.9740	0.0226	0.0083	0.9860	0.0102	0.0169	0.9895	0.0032

TABLE IX
PERFORMANCE METRICS OF DIFFERENT SEASONS
FOR THE DETROIT DATASET

Method	Time Step											
	Spring			Summer			Autumn			Winter		
	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE
Auto-LSTM	0.0322	0.9557	0.0241	0.0331	0.9568	0.0242	0.0324	0.9522	0.0239	0.0263	0.9548	0.0237
XGBF-DNN	0.0317	0.9569	0.0236	0.0326	0.9580	0.0237	0.0319	0.9534	0.0234	0.0258	0.9560	0.0232
LSTM	0.0341	0.9498	0.0223	0.0350	0.9509	0.0224	0.0343	0.9463	0.0221	0.0282	0.9489	0.0219
CLSTM	0.0338	0.9499	0.0239	0.0347	0.9510	0.0240	0.0340	0.9464	0.0237	0.0279	0.9491	0.0235
DWT-CLSTM	0.0337	0.9504	0.0204	0.0346	0.9514	0.0205	0.0339	0.9468	0.0202	0.0278	0.9495	0.0200
MEA-ANN	0.0252	0.9610	0.0199	0.0284	0.9614	0.0201	0.0273	0.9568	0.0195	0.0207	0.9595	0.0195
SCA-CLSTM	0.0222	0.9684	0.0194	0.0261	0.9688	0.0195	0.0243	0.9641	0.0190	0.0177	0.9669	0.0190
MSCA-CLSTM	0.0181	0.9759	0.0146	0.0213	0.9763	0.0147	0.0202	0.9717	0.0142	0.0136	0.9744	0.0142

case study that the number of kernel size between 3 and 7 is higher than all other kernel size ranges. The results obtained from these three violin plots show that the proposed MSCA model allocates the hyperparameter values to the CLSTM, which accounts for a smaller amount of computation when training the network.

Table VII shows the average time consumed based on optimization, training, and testing times for different

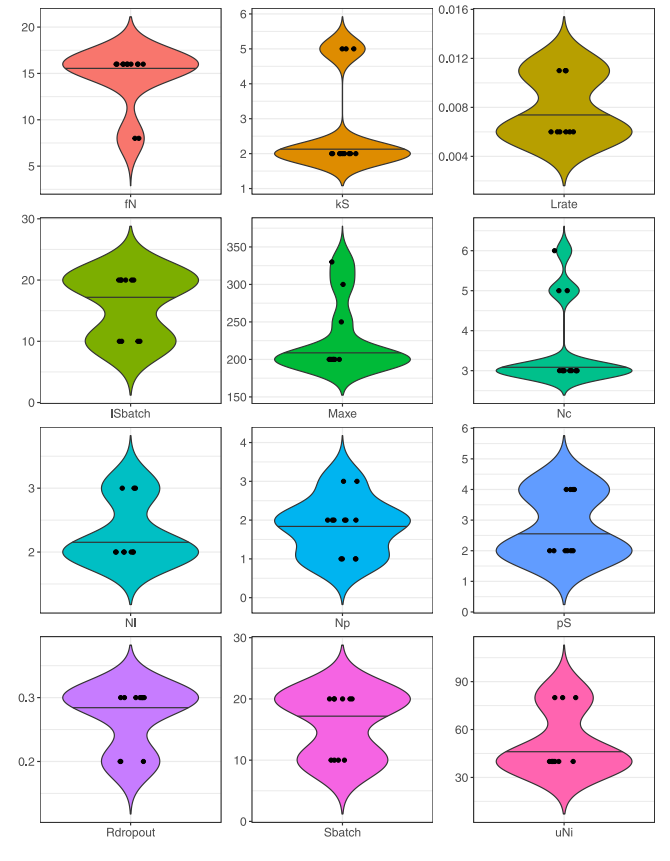


Fig. 12. Violin plots of the values obtained from the evolved hyperparameters applied in the MSCA-CLSTM algorithm for the Detroit case study of the 2-step ahead forecasting horizon.

TABLE X
PERFORMANCE METRICS OF DIFFERENT SEASONS
FOR THE SAN ANTONIO DATASET

Method	Time Step											
	Spring			Summer			Autumn			Winter		
	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE	RMSE	Pearson	MAE
Auto-LSTM	0.0322	0.9557	0.0241	0.0331	0.9568	0.0242	0.0324	0.9522	0.0239	0.0263	0.9548	0.0237
XGBF-DNN	0.0317	0.9569	0.0236	0.0326	0.9580	0.0237	0.0319	0.9534	0.0234	0.0258	0.9560	0.0232
LSTM	0.0341	0.9498	0.0223	0.0350	0.9509	0.0224	0.0343	0.9463	0.0221	0.0282	0.9489	0.0219
CLSTM	0.0338	0.9499	0.0239	0.0347	0.9510	0.0240	0.0340	0.9464	0.0237	0.0279	0.9491	0.0235
DWT-CLSTM	0.0337	0.9504	0.0204	0.0346	0.9514	0.0205	0.0339	0.9468	0.0202	0.0278	0.9495	0.0200
MEA-ANN	0.0252	0.9610	0.0199	0.0284	0.9614	0.0201	0.0273	0.9568	0.0195	0.0207	0.9595	0.0195
SCA-CLSTM	0.0222	0.9684	0.0194	0.0261	0.9688	0.0195	0.0243	0.9641	0.0190	0.0177	0.9669	0.0190
MSCA-CLSTM	0.0181	0.9759	0.0146	0.0213	0.9763	0.0147	0.0202	0.9717	0.0142	0.0136	0.9744	0.0142

forecasting algorithms. It should be noted that the Auto-LSTM, XGBF-DNN, LSTM, CLSTM, and DWT-CLSTM models do not employ any optimization algorithm in their overall procedures. Therefore, the optimization time of these models is not reported in Table VII. As we can see from this table, our proposed MSCA-CLSTM algorithm has the shortest time among the optimization-based algorithms (i.e., MEA-ANN and SCA-CLSTM) in terms of optimization time and also the lowest time consumption among other algorithms in terms of training and testing times.

To reflect the seasonality of GHI, we perform the proposed model and other benchmark models over three USA GHI datasets. It should be mentioned that we divide all three GHI datasets into four different seasons, allocating 75% of each dataset to training and the remaining 25% to testing. These findings are indicated in Tables VIII–X. As the results of these tables show, the proposed algorithm has the best performance

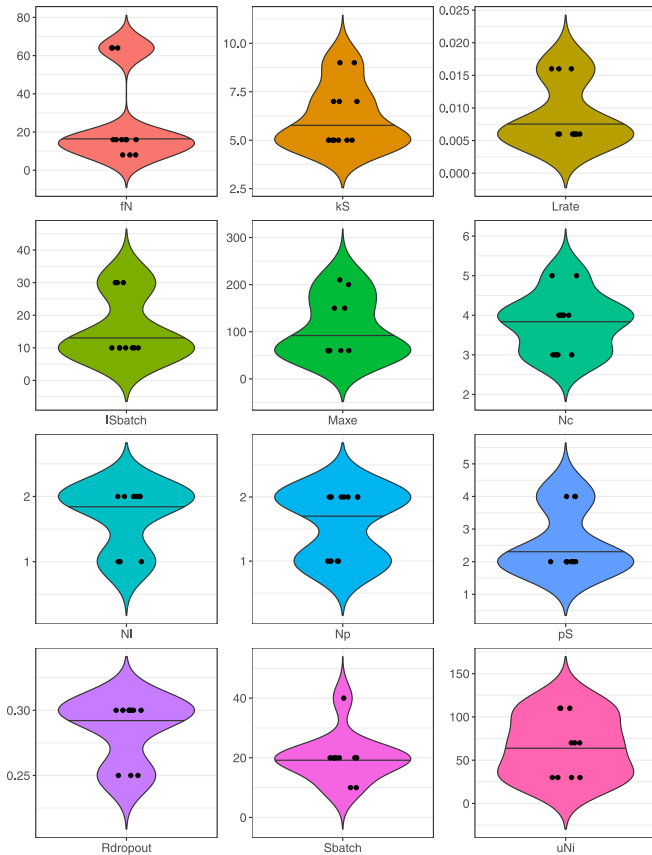


Fig. 13. Violin plots of the values obtained from the evolved hyperparameters applied in the MSCA-CLSTM algorithm for the San Antonio case study of the 2-step ahead forecasting horizon.

for all seasons in terms of three evaluation criteria. Therefore, it can be concluded that the proposed method not only performs effectively on the whole time-series data points but also it is very efficient for accurately forecasting the time series divided into different seasons.

IV. CONCLUSION

In this article, we have proposed a new deep learning framework called MSCA-CLSTM to predict the GHI solar irradiance. The proposed MSCA is a three-stage modification of its origin, which has been developed to define optimal hyperparameters for the deep CLSTM architectures. The efficiency of MSCA-CLSTM was verified through experiments on the Columbus, Detroit, and San Antonio case studies with the solar GHI under one-step and multistep-ahead forecasting scenarios. The GHI data from Columbus, Detroit, and San Antonio were used as case studies. We also applied the MI strategy to select the efficient input features for the deep CLSTM model. The proposed MSCA-CLSTM model outperformed other models in terms of MAE, RMSE, and Pearson metrics. For future studies, we plan on developing a more robust MSCA-CLSTM version in order to forecast more complex time-series forecasting problems with a higher level of demand projections. This will include wind power and electricity load forecasting to reduce the energy waste and loss.

REFERENCES

- [1] M. Egido and E. Lorenzo, "The sizing of stand alone PV-system: A review and a proposed new method," *Solar Energy Mater. Solar Cells*, vol. 26, nos. 1–2, pp. 51–69, 1992.
- [2] M.-H. Kim, D. Kim, J. Heo, and D.-W. Lee, "Techno-economic analysis of hybrid renewable energy system with solar district heating for net zero energy community," *Energy*, vol. 187, Nov. 2019, Art. no. 115916.
- [3] M. Brinkerink, B. O. Gallachóir, and P. Deane, "A comprehensive review on the benefits and challenges of global power grids and intercontinental interconnectors," *Renew. Sustain. Energy Rev.*, vol. 107, pp. 274–287, Jun. 2019.
- [4] C.-M. Cheng, S.-L. Tsao, and P.-Y. Lin, "SEEDS: A solar-based energy-efficient distributed server farm," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 45, no. 1, pp. 143–156, Jan. 2015.
- [5] A. C. Ferreira, J. Silva, S. Teixeira, J. C. Teixeira, and S. A. Nebra, "Assessment of the stirling engine performance comparing two renewable energy sources: Solar energy and biomass," *Renew. Energy*, vol. 154, pp. 581–597, Jul. 2020.
- [6] B. Robertson, J. Bekker, and B. Buckham, "Renewable integration for remote communities: Comparative allowable cost analyses for hydro, solar and wave energy," *Appl. Energy*, vol. 264, Apr. 2020, Art. no. 114677.
- [7] J. Lee and M. M. Shepley, "Benefits of solar photovoltaic systems for low-income families in social housing of Korea: Renewable energy applications as solutions to energy poverty," *J. Build. Eng.*, vol. 28, Mar. 2020, Art. no. 101016.
- [8] X. Kong, X. Liu, L. Ma, and K. Y. Lee, "Hierarchical distributed model predictive control of standalone wind/solar/battery power system," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 8, pp. 1570–1581, Aug. 2019.
- [9] H. Chen, H. Yi, B. Jiang, K. Zhang, and Z. Chen, "Data-driven detection of hot spots in photovoltaic energy systems," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 49, no. 8, pp. 1731–1738, Aug. 2019.
- [10] D. Yang and N. Chen, "Expanding existing solar irradiance monitoring network using entropy," *IEEE Trans. Sustain. Energy*, vol. 6, no. 4, pp. 1208–1215, Oct. 2015.
- [11] S. Salcedo-Sanz, C. Casanova-Mateo, J. Muñoz-Marí, and G. Camps-Valls, "Prediction of daily global solar irradiation using temporal Gaussian processes," *IEEE Geosci. Remote Sens. Lett.*, vol. 11, no. 11, pp. 1936–1940, Nov. 2014.
- [12] E. D. Obando, S. X. Carvajal, and J. P. Agudelo, "Solar radiation prediction using machine learning techniques: A review," *IEEE Latin Amer. Trans.*, vol. 17, no. 4, pp. 684–697, Apr. 2019.
- [13] G. Capizzi, C. Napoli, and F. Bonanno, "Innovative second-generation wavelets construction with recurrent neural networks for solar radiation forecasting," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 23, no. 11, pp. 1805–1815, Nov. 2012.
- [14] M. Kakimoto, Y. Endoh, H. Shin, R. Ikeda, and H. Kusaka, "Probabilistic solar irradiance forecasting by conditioning joint probability method and its application to electric power trading," *IEEE Trans. Sustain. Energy*, vol. 10, no. 2, pp. 983–993, Apr. 2019.
- [15] M. H. Rahman and S. Yamashiro, "Novel distributed power generating system of PV-ECaSS using solar energy estimation," *IEEE Trans. Energy Convers.*, vol. 22, no. 2, pp. 358–367, Jun. 2007.
- [16] E. Lorenz, J. Hurka, D. Heinemann, and H. G. Beyer, "Irradiance forecasting for the power prediction of grid-connected photovoltaic systems," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 2, no. 1, pp. 2–10, Mar. 2009.
- [17] C. Cui, Y. Zou, L. Wei, and Y. Wang, "Evaluating combination models of solar irradiance on inclined surfaces and forecasting photovoltaic power generation," *IET Smart Grid*, vol. 2, no. 1, pp. 123–130, 2019.
- [18] Y.-Y. Hong, J. J. F. Martinez, and A. C. Fajardo, "Day-ahead solar irradiance forecasting utilizing Gramian angular field and convolutional long short-term memory," *IEEE Access*, vol. 8, pp. 18741–18753, 2020.
- [19] M. H. Alsharif and M. K. Younes, "Evaluation and forecasting of solar radiation using time series adaptive neuro-fuzzy inference system: Seoul city as a case study," *IET Renew. Power Gener.*, vol. 13, no. 10, pp. 1711–1723, 2019.
- [20] J. Lee, W. Wang, F. Harrou, and Y. Sun, "Reliable solar irradiance prediction using ensemble learning-based models: A comparative study," *Energy Convers. Manage.*, vol. 208, Mar. 2020, Art. no. 112582.

- [21] K. Yan, H. Shen, L. Wang, H. Zhou, M. Xu, and Y. Mo, "Short-term solar irradiance forecasting based on a hybrid deep learning methodology," *Information*, vol. 11, no. 1, p. 32, 2020.
- [22] H. Jiang, Y. Gu, Y. Xie, R. Yang, and Y. Zhang, "Solar irradiance capturing in cloudy sky days—A convolutional neural network based image regression approach," *IEEE Access*, vol. 8, pp. 22235–22248, 2020.
- [23] F. Wang *et al.*, "Generative adversarial networks and convolutional neural networks based weather classification model for day ahead short-term photovoltaic power forecasting," *Energy Convers. Manage.*, vol. 181, pp. 443–462, Feb. 2019.
- [24] F. Wang, Z. Xuan, Z. Zhen, K. Li, T. Wang, and M. Shi, "A day-ahead PV power forecasting method based on LSTM-RNN model and time correlation modification under partial daily pattern prediction framework," *Energy Convers. Manage.*, vol. 212, May 2020, Art. no. 112766.
- [25] M. Husein and I.-Y. Chung, "Day-ahead solar irradiance forecasting for microgrids using a long short-term memory recurrent neural network: A deep learning approach," *Energies*, vol. 12, no. 10, p. 1856, 2019.
- [26] F. Wang *et al.*, "A minutely solar irradiance forecasting method based on real-time sky image-irradiance mapping model," *Energy Convers. Manage.*, vol. 220, Sep. 2020, Art. no. 113075.
- [27] Z. Zhen *et al.*, "Deep learning based surface irradiance mapping model for solar PV power forecasting using sky image," *IEEE Trans. Ind. Appl.*, vol. 56, no. 4, pp. 3385–3396, Jul./Aug. 2020.
- [28] S. Ahmadian and A. R. Khantemouri, "Training back propagation neural networks using asexual reproduction optimization," in *Proc. 7th Conf. Inf. Knowl. Technol. (IKT)*, Urmia, Iran, 2015, pp. 1–6.
- [29] S. M. J. Jalali, S. Ahmadian, P. M. Kebria, A. Khosravi, C. P. Lim, and S. Nahavandi, "Evolving artificial neural networks using butterfly optimization algorithm for data classification," in *Proc. Int. Conf. Neural Inf. Process. (ICONIP)*, 2019, pp. 596–607.
- [30] S. M. J. Jalali, S. Ahmadian, A. Khosravi, S. Mirjalili, M. R. Mahmoudi, and S. Nahavandi, "Neuroevolution-based autonomous robot navigation: A comparative study," *Cogn. Syst. Res.*, vol. 62, pp. 35–43, Aug. 2020.
- [31] S. M. J. Jalali *et al.*, "Towards novel deep neuroevolution models: Chaotic levy grasshopper optimization for short-term wind speed forecasting," *Eng. Comput.*, vol. 1, pp. 1–25, Mar. 2021.
- [32] S. J. Mousavirad, S. M. J. Jalali, S. Ahmadian, A. Khosravi, G. Schaefer, and S. Nahavandi, "Neural network training using a biogeography-based learning strategy," in *Proc. Int. Conf. Neural Inf. Process.*, 2020, pp. 147–155.
- [33] S. M. J. Jalali, S. Ahmadian, A. Khosravi, M. Shafie-Khah, S. Nahavandi, and J. P. S. Catalao, "A novel evolutionary-based deep convolutional neural network model for intelligent load forecasting," *IEEE Trans. Ind. Informat.*, early access, Mar. 12, 2021, doi: [10.1109/TII.2021.3065718](https://doi.org/10.1109/TII.2021.3065718).
- [34] S. Mirjalili, "SCA: A sine cosine algorithm for solving optimization problems," *Knowl. Based Syst.*, vol. 96, pp. 120–133, Mar. 2016.
- [35] F. Wang, Y. Yu, Z. Zhang, J. Li, Z. Zhen, and K. Li, "Wavelet decomposition and convolutional LSTM networks based improved deep learning model for solar irradiance forecasting," *Appl. Sci.*, vol. 8, no. 8, p. 1286, 2018.
- [36] Y. Feng, D. Gong, Q. Zhang, S. Jiang, L. Zhao, and N. Cui, "Evaluation of temperature-based machine learning and empirical models for predicting daily global solar radiation," *Energy Convers. Manage.*, vol. 198, Oct. 2019, Art. no. 111780.
- [37] A. Gensler, J. Henze, B. Sick, and N. Raabe, "Deep learning for solar power forecasting—An approach using autoencoder and LSTM neural networks," in *Proc. IEEE Int. Conf. Syst. Man Cybern. (SMC)*, Budapest, Hungary, 2016, pp. 002858–002865.
- [38] P. Kumari and D. Toshniwal, "Extreme gradient boosting and deep neural network based ensemble learning approach to forecast hourly solar irradiance," *J. Clean. Prod.*, vol. 279, Jan. 2021, Art. no. 123285.
- [39] X. Qing and Y. Niu, "Hourly day-ahead solar irradiance prediction using weather forecasts by LSTM," *Energy*, vol. 148, pp. 461–468, Apr. 2018.
- [40] H. Zang, L. Liu, L. Sun, L. Cheng, Z. Wei, and G. Sun, "Short-term global horizontal irradiance forecasting based on a hybrid CNN-LSTM model with spatiotemporal correlations," *Renew. Energy*, vol. 160, pp. 26–41, Nov. 2020.
- [41] M. Sengupta, Y. Xie, A. Lopez, A. Habte, G. Maclaurin, and J. Shelby, "The national solar radiation data base (nsrdb)," *Renew. Sustain. Energy Rev.*, vol. 89, pp. 51–60, Jun. 2018.
- [42] M. Khodayar, J. Wang, and M. Manthouri, "Interval deep generative neural network for wind speed forecasting," *IEEE Trans. Smart Grid*, vol. 10, no. 4, pp. 3974–3989, Jul. 2019.



Seyed Mohammad Jafar Jalali (Member, IEEE) received the B.S. degree in information technology engineering from Fasa University, Fasa, Iran, in 2014, and the M.S. degree in information technology major in advanced information systems from Allameh Tabataba'i University, Tehran, Iran, in 2016. He is currently pursuing the Ph.D. degree in artificial intelligence with the Institute for Intelligent Systems Research and Innovation, Deakin University, Waurn Ponds, VIC, Australia.

In 2017, he was a Research Assistant with the University of Massachusetts, Amherst, MA, USA, conducting research in the field of artificial intelligence. His primary research interests include machine learning, deep neural architecture search, deep learning, and optimization.

Mr. Jalali received the prestigious Deakin University Postgraduate Research Scholarship in 2018.



Sajad Ahmadian received the B.S. degree in computer engineering from Razi University, Kermanshah, Iran, in 2011, and the M.S. degree in computer engineering, artificial intelligence from the University of Kurdistan, Sanandaj, Iran, in 2014. He is currently pursuing the Ph.D. degree in computer engineering, artificial intelligence with the University of Zanjan, Zanjan, Iran.

He conducted a part of his Ph.D. research work with the Laboratory of Complex Networks, RMIT University, Melbourne, VIC, Australia, from February 2018 to July 2018. He is an Assistant Professor with the Faculty of Information Technology, Kermanshah University of Technology, Kermanshah. His research interests include recommender systems, social networks analysis, data mining, and machine learning.



Abdollah Kavousi-Fard (Senior Member, IEEE) received the B.Sc. degree in electrical engineering from the Shiraz University of Technology, Shiraz, Iran, in 2009, the M.Sc. degree in electrical engineering from Shiraz University, Shiraz, in 2011, and the Ph.D. degree in electrical engineering from the Shiraz University of Technology in 2016.

He was a Postdoctoral Research Assistant with the University of Michigan, Ann Arbor, MI, USA, from 2016 to 2018. He was a Researcher with the University of Denver, Denver, CO, USA, from 2015 to 2016, conducting research on microgrids. He is currently an Assistant Professor with the Shiraz University of Technology. His current research interests include operation, management, and cybersecurity analysis of smart grids, microgrid, smart city, electric vehicles, as well as protection of power systems, reliability, artificial intelligence, and machine learning.



Abbas Khosravi (Senior Member, IEEE) received the B.Sc. degree in electrical engineering from the Sharif University of Technology, Tehran, Iran, in 2002, the M.Sc. degree in electrical engineering from the Amirkabir University of Technology, Tehran, in 2005, and the Ph.D. degree in artificial intelligence from Deakin University, Waurn Ponds, VIC, Australia, in 2010.

He is currently an Associate Professor with the Institute for Intelligent Systems Research and Innovation, Deakin University. His research interests include artificial intelligence, data mining, and optimization.



Saeid Nahavandi (Fellow, IEEE) received the Ph.D. degree in artificial intelligence from Durham University, Durham, U.K., in 1991.

He is an Alfred Deakin Professor, a Pro-Vice-Chancellor, a Chair of Engineering, and the Founding Director of the Institute for Intelligent Systems Research and Innovation, Deakin University, Waurn Ponds, VIC, Australia. He has published more than 950 scientific papers in various international journals and conferences. His research interests include the modeling of complex systems, robotics, and haptics.

Dr. Nahavandi is a Fellow of the Australian Academy of Technological Sciences and Engineering, Engineers Australia, and the Institution of Engineering and Technology.